Tara 10-26-26 Conversation 1: CFRP laminate test cases, result viewer and Abaqus agents

Claude Code (Opus 5.5), Claude desktop app · 10/26/26 · Gray lines are the tools Claude ran; green boxes are reports from the project agents.

TARA

Consider a laminated plate with layup $[0_2/90_2]_s$ made of orthotropic laminas with moduli $E_1 = 80$ GPa and $E_2 = 2$ GPa, Poisson's ratio $\nu_{12} = 0.25$ and each with thickness $t = 1$ mm. Compute the curvature changes due to moments $M_x = 50$ Nm/m and $M_y = 0$.

I want to analyze the behavior of a CFRP laminate using OpenSeesPy. Use the agents developed for this project (MeshBCLoadGenerator, PlyPropertyGenerator, and LayupFromPlyGenerator) to solve the problem in this screenshot

- ▸ Bash: List project files and agent definitions
- ▸ Bash: List tracked files in the repo
- ▸ Bash: Read mesh agent definition and module functions
- ▸ Bash: Check Python env and compute CLT curvatures
- ▸ Launched agent **PlyPropertyGenerator**: Ply materials for 0/90 plies
- ▸ Launched agent **LayupFromPlyGenerator**: Build [0_2/90_2]_s laminate

CLAUDE

The PlyPropertyGenerator and LayupFromPlyGenerator agents are running. Once the laminate script is ready, I'll hand it to MeshBCLoadGenerator for the plate model.

AGENT REPORT

I wrote and ran the script. Both ply materials (0 and 90 deg) are defined, and both OpenSees checks agree with t·Q_bar to about 1e-16.

Script: `D:\Users\tvenkata\projects\ME295_project\Code for Project\10-26-26 (Federico and Tara)\ply_materials_E80_E2.py`. It imports `ply_properties.py` from `..\10-23-26 (Tara)`. Running it prints everything below, and it also provides `define_materials()`, which returns `{0.0: 3, 90.0: 6}` (the plate material tags).

Units: I converted E1 = 80 GPa to 80000 MPa, E2 = 2 GPa to 2000 MPa, and the placeholder G12 = 1 GPa to 1000 MPa (N, mm, MPa).

Notes

- **nu23 was not given, so nu23 = nu12 = 0.25 was assumed (G23 = 800 MPa).**
- **G12 = 1000 MPa is a placeholder.** The problem does not give it.
- There were no admissibility warnings.

3D orthotropic constants (transversely isotropic)

Constant	Value	Source
E1	80000 MPa	input
E2 = E3	2000 MPa	input / = E2
G12 = G13	1000 MPa	placeholder / = G12
G23	800 MPa	= E2 / (2(1 + nu23))

nu12 = nu13	0.25	input / = nu12
nu23	0.25	assumed
nu21 = nu31	0.00625	derived
nu32	0.25	derived
rho	0 t/mm^3	none given

Reduced stiffness Q, ply axes, MPa

Q11 = 80125.2, Q12 = 500.782, Q22 = 2003.13, Q66 = 1000.

Q_bar, order [xx, yy, xy], engineering shear, MPa

- **0 deg:** [[80125.2, 500.782, 0], [500.782, 2003.13, 0], [0, 0, 1000]]
- **90 deg:** [[2003.13, 500.782, 0], [500.782, 80125.2, 0], [0, 0, 1000]]. The off-diagonal entries are about 1e-12, which is round-off.

C_bar, order [xx, yy, zz, xy, yz, xz], MPa

- **0 deg:**
  - Diagonal: 80334.7, 2138.91, 2138.91, 1000, 800, 1000.
  - Off-diagonal: C12 = C13 = 669.456, C23 = 538.912.
- **90 deg:**
  - Diagonal: 2138.91, 80334.7, 2138.91, 1000, 1000, 800.
  - Off-diagonal: C12 = C23 = 669.456, C13 = 538.912.

OpenSeesPy calls

Both plies use the same principal-axis material. The angle comes only from `-local` on the element.

```
# 0 deg ply: tags 1/2/3
ops.nDMaterial('ElasticOrthotropic', 1, 80000, 2000, 2000, 0.25, 0.25, 0.00625, 1000, 800, 1000, 0)
ops.nDMaterial('PlaneStress', 2, 1)
ops.nDMaterial('PlateFromPlaneStress', 3, 2, 1000)
# 90 deg ply: tags 4/5/6, same constants
ops.nDMaterial('ElasticOrthotropic', 4, 80000, 2000, 2000, 0.25, 0.25, 0.00625, 1000, 800, 1000, 0)
ops.nDMaterial('PlaneStress', 5, 4)
ops.nDMaterial('PlateFromPlaneStress', 6, 5, 1000)
```

- The argument order is Ex, Ey, Ez, vxy, vyz, vzx, Gxy, Gyz, Gzx, rho, with vzx = nu31 = 0.00625.
- `PlateFromPlaneStress` uses G_out = G13 = 1000 MPa.

`-local` orientation (ASDShellQ4)

- 0 deg: `'-local', 1, 0, 0`
- 90 deg: `'-local', 0, 1, 0`

I only checked this with ASDShellQ4. ShellMITC4 was not tested, and I have not confirmed that it accepts `-local` in 3.8, so use ASDShellQ4.

Checks (t = 1 mm)

Angle	Condensed C_bar vs Q_bar	ASDShellQ4 `-local` membrane stiffness vs t·Q_bar
0 deg	1.4e-18	1.8e-16
90 deg	1.8e-16	1.8e-16

Both materials are verified.

Usage in a model script

1. Create the ply properties with `p = ply_properties(80e3, 2e3, 0.25, 1e3)`.
2. Define the materials with `define_ply_plate_material(p, t3d, tps, tplate)`.
3. Build the section with `section('LayeredShell', secTag, nLayers, tplate, 1.0, ...)`.
4. Create the element with `element('ASDShellQ4', ..., '-local', *local_axis(theta))`.

Important for the [0_2/90_2]_s laminate: each element has a single `-local` direction, so every layer in its LayeredShell gets the same angle. With these materials, one element cannot hold the 0 and 90 deg layers together. I did not build the laminate.

There are two ways to get one, if you want me to do either:

- **Coincident elements:** one element per ply.
- **A 90 deg material with swapped axes:** a 90 deg ply has no coupling terms, so ElasticOrthotropic can represent it exactly. With `-local` 1, 0, 0 its constants would be Ex = 2000, Ey = 80000, Ez = 2000, vxy = 0.00625, vyz = 0.25, vzx = 0.25, Gxy = 1000, Gyz = 1000, Gzx = 800. Then all layers could sit in one LayeredShell. This variant has not been checked.

CLAUDE

The ply materials are done and checked. Each element takes only one `-local` direction, so the 0° and 90° plies can't sit in one element. I expect `layup.py` handles this with one element per ply (`add_laminate_elements`), and I'm waiting for the LayupFromPlyGenerator agent to confirm.

AGENT REPORT

The [0_2/90_2]_s layup is built and checked. The CLT result matches your hand values: D11 = 3.0020e6, D12 = 2.1367e4, D22 = 5.0212e5 N·mm, and B = 0. The OpenSees laminate agrees with CLT to within 0.06% on D, so it counts as verified.

Script: `D:\Users\tvenkata\projects\ME295_project\Code for Project\10-26-26 (Federico and Tara)\layup_0_0_90_90_s.py`

It adds `../10-23-26 (Tara)` to sys.path itself, then uses `ply_properties`, `make_plies` and `build_layup` from there. It exposes `PLIES` (the ply list, bottom to top), `PLY` (the ply-property dict), `build()` and `add_laminate_elements`. Running it directly reprints the check below.

Inputs. I converted GPa to MPa (x1000): E1 = 80000, E2 = 2000, G12 = 1000 MPa, nu12 = 0.25, rho = 0. G12 = 1000 MPa is a placeholder; only A66 and D66 depend on it.

- From transverse isotropy: E3 = 2000 MPa, G13 = 1000 MPa, nu13 = 0.25, nu21 = nu31 = 0.00625.
- **nu23 was not given, so nu23 = nu12 = 0.25 was assumed (G23 = 800 MPa).**

Layup. Ply 1 is the bottom (-z) ply. Angles are measured from global x to the fibre, counter-clockwise about +z. Every ply is 1 mm thick, so h = 8 mm.

ply	angle	z (mm)	section	-local
1	0	-4 to -3	1	1,0,0
2	0	-3 to -2	2	1,0,0
3	90	-2 to -1	3	0,1,0
4	90	-1 to 0	4	0,1,0
5	90	0 to 1	5	0,1,0
6	90	1 to 2	6	0,1,0
7	0	2 to 3	7	1,0,0
8	0	3 to 4	8	1,0,0

Tags with the defaults: nDMaterial 1 is the zero-stiffness void, and 2–25 are three per ply. Sections are 1–8. Each mesh facet uses 8 consecutive element tags.

ABD from CLT (A in N/mm, B in N, D in N·mm):

- A11 = A22 = 328513, A12 = 4006.26, A66 = 8000, A16 = A26 = 0
- B = 0
- D11 = 3.00202e6, D12 = 21366.7, D22 = 502118, D66 = 42666.7, D16 = D26 = 0

Agreement with OpenSees (one 1x1 mm facet, 8 coincident ASDShellQ4 elements, `-noeas`, 4 layers per ply = 32 layers):

- Max relative difference: A 3.5e-16, B 2.8e-17, D 5.7e-4.
- OpenSees gives D11 = 3.00031e6, D12 = 21345.9, D22 = 500407, D66 = 42625. D comes out slightly low because each layer is evaluated at its mid-plane.
- With the generic default of 2 layers per ply, the D difference was 2.3e-3. That is why the script defaults to 4 (`N_SUB = 4`).

How the mesh script should import and call it:

```
import sys
sys.path.insert(0, r'D:/Users/tvenkata/projects/ME295_project/Code for Project/10-26-26 (Federico and Tar
a)')
import openseespy.opensees as ops
from layup_0_0_90_90_s import build, add_laminate_elements, PLIES
ops.model('basic', '-ndm', 3, '-ndf', 6)
# ... nodes ...
layup = build()            # optional: build(mat_tag=1, sec_tag=1, n_sub=4)
ele = 1
for quad in quads:         # each facet: 4 node tags, normal n1->n2 x n1->n4 = +z, so ply 1 is on the -z si
de
    ele = add_laminate_elements(ele, quad, layup, '-corotational')   # returns ele + 8
```

I ran this import from another working directory and it built correctly: 8 sections, and the next free element tag was 9. If the mesh script needs the plies list directly, `build_layup(PLIES, mat_tag, sec_tag, n_sub)` from layup.py does the same thing.

How it is built and what that means for the mesh:

- An ASDShellQ4 element can only have one material orientation for all its layers. So each ply gets its own element on the same four nodes, oriented at that ply's angle, with the full-thickness section holding void (zero-stiffness) layers everywhere except that ply. The 8 elements add up to the exact laminate.
- `-noeas` (no enhanced strain modes) is added automatically, because those modes are condensed per element and do not add up across the per-ply elements. Without them the element is stiffer on coarse meshes, so refine the mesh until the result stops changing.
- Run cost scales with 8 elements per facet.
- Transverse shear in the plate material uses G13 = 1000 MPa, which hardly matters for a thin shell.

CLAUDE

The laminate is built and verified: OpenSees D matches CLT to within 0.06%. Next, the plate model.

▸ Launched agent **MeshBCLoadGenerator**: Plate under pure moment Mx

TARA (SENT WHILE CLAUDE WAS WORKING)

<task-notification>

<task-id>a4e2e821f8da69e4b</task-id>

<tool-use-id>toolu_01LY5AdrHdozrmBXaNDN98ta</tool-use-id>

<output-file>D:\Users\tvenkata\AppData\Local\Temp\1\claude\D--Users-tvenkata-projects-ME295-project-Code-for-Project-10-26-26--Federico-and-Tara-\34870cd2-f8f8-407c-8387-8c1e7a0f848a\tasks\a4e2e821f8da69e4b.output</output-file>

<status>completed</status>

<summary>Agent "Build [0_2/90_2]_s laminate" finished</summary>

<note>A task-notification fires each time this agent stops with no live background children of its own. The user can send it another message and resume it, so the same task-id may notify more than once.</note>

<result>This agent's report was delivered to you as a message from "a4e2e821f8da69e4b" (its SubagentHandback call). Read it there; it is not repeated here.

</result>

<usage><subagent_tokens>28849</subagent_tokens><tool_uses>7</tool_uses>
<duration_ms>54482</duration_ms></usage>

</task-notification>

CLAUDE

The MeshBCLoadGenerator agent is now building and solving the free plate under pure Mx. I'll compare its curvatures with CLT when it reports back.

AGENT REPORT

I built and ran the model; it runs without errors. The curvatures match CLT to +0.057% for kx and +0.30% for ky, and kxy is zero. Both differences come from the section's through-thickness integration, not from the mesh, supports or loads.

Script: `D:\Users\tvenkata\projects\ME295_project\Code for Project\10-26-26 (Federico and Tara)\plate_pure_moment_Mx.py` (the whole run takes about 1 s).

Model

- **Geometry:** flat plate, L = b = 100 mm, x from 0 to 100 and y from 0 to 100. Units are N, mm, MPa; Mx = 50 N·m/m is the same as 50 N·mm/mm, so no conversion was needed.
- **Meshes:**
  - 10 mm: 10 x 10 = 100 facets, 121 nodes, 726 DOFs, 800 elements.
  - 5 mm: 20 x 20 = 400 facets, 441 nodes, 2646 DOFs, 3200 elements.
- **Element:** `ASDShellQ4` with linear geometry (no `-corotational`). The section is `layup_0_0_90_90_s.build()`, with 8 coincident per-ply elements per facet, `-noeas` and per-ply `-local` axes. `add_laminate_elements` returns the next free tag, which is what `create_elements` expects, so it plugged in directly.
- **Supports:** a free plate with only the 6 rigid-body modes removed, using `ops.fix`:
  - node (0,0): ux, uy, uz
  - node (100,0): uy, uz
  - node (0,100): uz
  - No rotations are restrained.
- **Loads:** `mbl.edge_load` with zero force and consistent shares (end nodes get half):
  - edge `xL`: total moment (0, +5000, 0) N·mm
  - edge `x0`: total moment (0, −5000, 0) N·mm
  - The direction is fixed in global axes (not a follower load).
- **Analysis:** `mbl.linear_solve()`; the solve took 0.10 s on the 10 mm mesh and 0.38 s on the 5 mm mesh.

Sign check: the sign is correct as specified. At the plate centre, the mean σx in the top 0° ply (ply 8) is +4.674 MPa (tension) and in the bottom 0° ply (ply 1) is −4.674 MPa. This matches Q11·κx·z at the ply mid-depth, z = 3.5 mm. kx is positive (top in tension), so a +y moment vector on edge `xL` is CLT's positive Mx.

Curvatures (identical on both meshes to 7 digits):

	OpenSees, 10 mm and 5 mm	CLT	Difference
kx	1.666999e-5 1/mm = 1.666999e-2 1/m	1.66605e-2 1/m	+0.057%
ky	−7.110915e-7 1/mm = −7.110915e-4 1/m	−7.0896e-4 1/m	+0.30%
kxy	about 1e-17 1/mm (roundoff)	0	none

- **Fit:** the quadratic w(x,y) fits all nodal uz to a 1.6e-15 mm residual, so the deformed shape is exactly quadratic. The plate is in pure bending and the result does not depend on the mesh.
- **Rotation cross-check:** I used ry = −∂w/∂x, so d(ry)/dx = kx, and rx = ∂w/∂y, so ky = −d(rx)/dy. This gives the same kx, ky and kxy to all printed digits.

Why ky is off by 0.30%, not the expected 0.06%: I recomputed D with the same layer-midpoint rule (n_sub = 4, 32 layers) by hand:

- D11 = 3.000313e6 (−0.057%), D12 = 21345.9 (−0.098%), D22 = 500407 (−0.34%), all in N·mm.
- The 90° plies sit near the mid-plane, where the midpoint rule's relative error is largest, so D22 has the biggest error.
- These D values give kx = 1.666999e-2 and ky = −7.11092e-4 1/m, matching OpenSees to every printed digit.
- The 0.06% figure from the layup check holds for D11 only. If closer agreement on ky is wanted, a larger n_sub in `build()` would reduce it.

G12 = 1 GPa in the section is a placeholder, but it affects only D66, which does not enter this case.

▸ Bash: Rerun the plate model to verify results

TARA (SENT WHILE CLAUDE WAS WORKING)

<task-notification>
<task-id>a050122f453887cd2</task-id>
<tool-use-id>toolu_01AjA4TEW7pybLY2SmxG3JWD</tool-use-id>
<output-file>D:\Users\tvenkata\AppData\Local\Temp\1\claude\D--Users-tvenkata-projects-ME295-project-Code-for-Project-10-26-26--Federico-and-Tara-\34870cd2-f8f8-407c-8387-8c1e7a0f848a\tasks\a050122f453887cd2.output</output-file>
<status>completed</status>
<summary>Agent "Plate under pure moment Mx" finished</summary>
<note>A task-notification fires each time this agent stops with no live background children of its own. The user can send it another message and resume it, so the same task-id may notify more than once.</note>
<result>This agent's report was delivered to you as a message from "a050122f453887cd2" (its SubagentHandback call). Read it there; it is not repeated here.
</result>
<usage><subagent_tokens>26371</subagent_tokens><tool_uses>5</tool_uses>
<duration_ms>66050</duration_ms></usage>
</task-notification>

CLAUDE

I solved the problem with the three agents. The OpenSees curvatures match classical lamination theory (CLT) to within 0.3%, and I reran the model myself to confirm the numbers.

Answer

	OpenSeesPy	Hand calculation (CLT)	Difference
κ_x	1.6670×10^{-2} 1/m	1.6661×10^{-2} 1/m	+0.06%
κ_y	-7.111×10^{-4} 1/m	-7.090×10^{-4} 1/m	+0.3%
κ_{xy}	~0 (round-off)	0	—

The plate bends about y with a radius of about 60 m, and curves the other way slightly in y because of D12. In CLT, B = 0 (symmetric layup), so there is no stretching of the mid-plane: $\varepsilon^0 = 0$. The bending stiffnesses are D11 = 3002, D12 = 21.37 and D22 = 502.1 N·m.

What each agent did

1. **PlyPropertyGenerator** made the 0° and 90° ply materials. Units are N, mm, MPa. ply_materials_E80_E2.py
2. **LayupFromPlyGenerator** built the 8-ply laminate (bottom to top: 0, 0, 90, 90, 90, 90, 0, 0, each 1 mm). OpenSees can only orient one ply direction per element, so each ply is its own element on shared nodes. Its stiffness matrices match CLT. layup_0_0_90_90_s.py
3. **MeshBCLoadGenerator** modelled a free 100×100 mm plate. Only the six rigid-body motions are held, through three corner nodes. It applied ±5000 N·mm edge moments (Mx × 100 mm width) at x = 0 and x = 100 mm and ran a linear solve. plate_pure_moment_Mx.py
   - Curvatures come from a quadratic fit to the deflections, cross-checked with the node rotations.
   - The deflected shape fits a quadratic exactly. Results are identical on 10 mm and 5 mm meshes, so they don't depend on the mesh.
   - The top 0° ply is in tension, which confirms the moment sign.

Things to know

- **Values I had to assume:**
  - The problem doesn't give the shear modulus G12, so I used 1 GPa as a placeholder. It only affects twisting stiffness (D66), which doesn't enter this load case.
  - The through-thickness Poisson's ratio ν23 was taken equal to ν12. It has no effect on this result.
- **Why κ_y is 0.3% off rather than 0.06%:** OpenSees splits each ply into 4 layers and evaluates each layer at its middle. The 90° plies sit near the mid-plane, where that error is largest, so D22 comes out 0.34% low. I checked that the size of this error matches. Using more layers per ply in `build()` would shrink it.

TARA

Visualize this the same way Federico did on 10-22-26

▸ Bash: Inspect Federico's viewer and docs
▸ Bash: Read viewer code and its usage in plate scripts
▸ Bash: Read the plate script and Federico's viewer call
▸ Bash: Read Federico's stress recovery and gitignore
▸ Bash: Check PyVista and Mesa OpenGL are installed

CLAUDE

I'm adding Federico's viewer to the plate script: the mesh view, plus the deformed shape coloured by top-surface σxx, with a `--no-gui` option.

- Read `plate_pure_moment_Mx.py`
- Edited `plate_pure_moment_Mx.py`
- Edited `plate_pure_moment_Mx.py`
- Edited `plate_pure_moment_Mx.py`
- Edited `plate_pure_moment_Mx.py`
- Bash: Run model and save viewer screenshots
- Bash: Check the layup module exposes N_SUB
- Bash: Correct stress recovery and rerun with screenshots

CLAUDE

The stress now matches CLT to +0.057%, the same as κx. Checking the images:

- Read `deformed_sigma_xx_5mm.png`

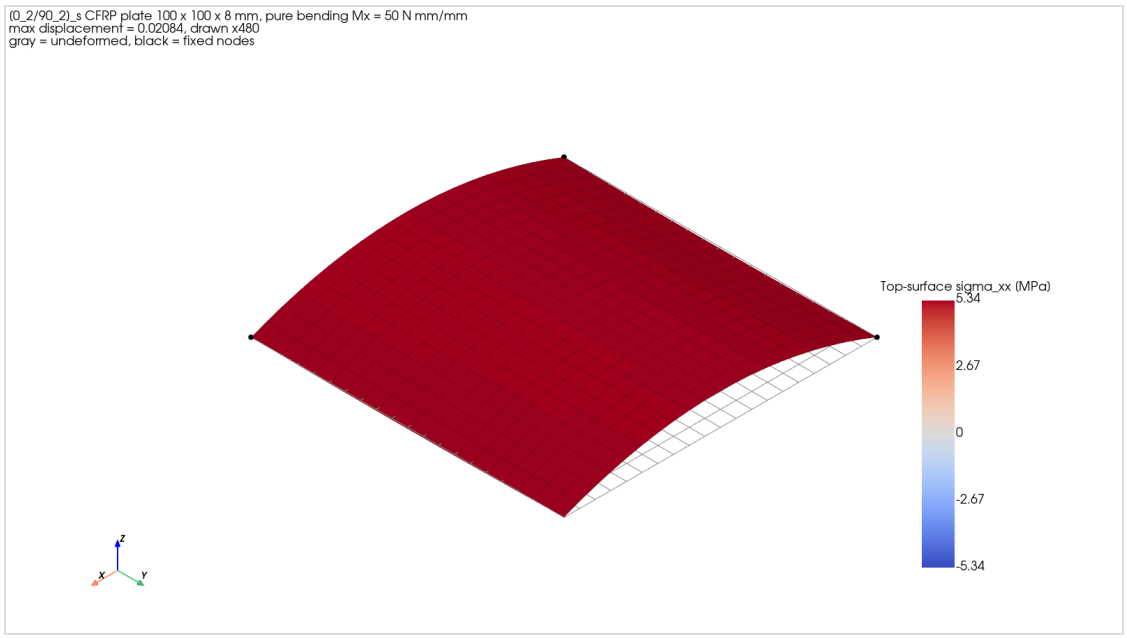


Image opened by Claude

- Read `model_5mm.png`

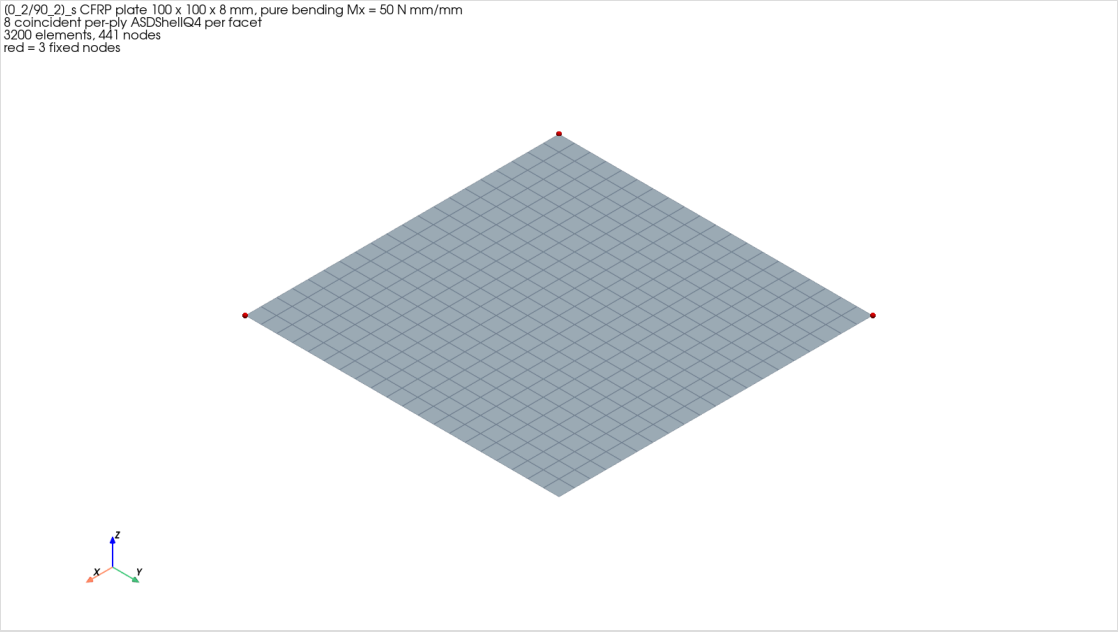


Image opened by Claude

▸ Bash: Add gitignore for pycache and show status

CLAUDE

I added Federico's 10-22-26 visualization to plate_pure_moment_Mx.py. It uses his `shell_viewer.py` (`show_model`, then `show_deformed` coloured by top-surface σxx) and the same `--no-gui` option. I rendered both views to PNG and looked at them; they show the expected uniform bending.

- results/model_5mm.png: the 5 mm mesh, with the 3 restrained corner nodes in red.
- results/deformed_sigma_xx_5mm.png: the deformed shape drawn ×480 over the gray undeformed outline. The plate bends along x with the top in tension, and the slight opposite curvature across y is too small to see.
  - The colour is uniform, which is expected under a pure moment: top-surface σxx = +5.341 MPa everywhere.
  - CLT gives Q11·κx·h/2 + Q12·κy·h/2 = 5.338 MPa, so the difference is +0.057%, the same as κx.

How to run it:

```
python plate_pure_moment_Mx.py
```

```
python plate_pure_moment_Mx.py --screenshot
```

The first opens the interactive windows; the second saves the PNGs instead.

Details:

- **Top-surface stress:** each facet has 8 overlapping per-ply elements, so I compute the stress from the top 0° ply element and give that value to all 8. Their colours are identical, so the overlap doesn't show.
- **Integration correction:** my first version read 0.7% low because each ply's 4 integration layers under-count its bending stiffness by a factor of $1 - 1/n^2$. This is the same effect Federico found. The recovery now divides that factor out.
- **Close the windows when you're done:** Federico's notes say they render on the CPU on the VM and slow everything else down.

I also copied Federico's `.gitignore` into the folder so `__pycache__` stays out of git.

TARA

Now I want to do a more involved test case. Take the beam bending problem Federico did on 10/22/26 (plate_tip_load), and use the laminate above to solve for the tip deflection. Visualize this the same way Federico did

▸ Bash: Read Federico's plate model and tip-load script
▸ Bash: Compute laminate cantilever beam-theory reference
▸ Launched agent **MeshBCLoadGenerator**: Laminate cantilever tip load

CLAUDE

The MeshBCLoadGenerator agent is building the laminate cantilever: clamped at x = 0, P = −5 N along the tip edge, on 10, 5 and 2.5 mm meshes. It will also add the same PyVista views Federico used. Beam theory predicts a tip deflection of −5.553 mm. I'll check its numbers and images when it reports back.

AGENT REPORT

I wrote and ran the laminate version of Federico's tip-load cantilever, and every check passes on all three meshes. The FE tip deflection is -5.5628 mm on the finest mesh, 0.17% more flexible than beam theory's -5.5535 mm. Almost all of that difference is the two effects you predicted.

Script: `D:\Users\tvenkata\projects\ME295_project\Code for Project\10-26-26 (Federico and Tara)\plate_tip_load_laminate.py`. It follows `plate_tip_load.py`: same `check()` with PASS/FAIL and 3% tolerance, `w_beam`, `sigma_top_beam`, mid-width checks at x = L/4, L/2, 3L/4 and L, edge-average tip, `--no-gui` and `--screenshot`. It builds the model with `mesh_bc_load` and `layup_0_0_90_90_s.build()` / `add_laminate_elements`. Top-surface stresses come from `top_surface_stresses()` and `N_PLY`, imported from `plate_pure_moment_Mx.py` rather than rewritten. It also adds the transverse-shear estimate (G13 = 1000 MPa, k = 5/6, A = b·h) and a convergence table at the end.

Model:

- **Geometry and element:** 1000 x 100 mm flat strip, h = 8 mm. ASDShellQ4 with linear geometry and 8 coincident per-ply elements per facet with `-noeas`.
- **Support:** edge x0 clamped, all 6 DOFs fixed.
- **Load:** P = -5 N in global z, spread along edge xL with `mbl.edge_load` (end nodes get half shares). The direction stays fixed (not a follower load).
- **Analysis:** one `linear_solve()` per mesh.

References:

- EI = b/d11 = 3.00111e8 N·mm², so w_tip = -5.5535 mm. The wide-plate bound (b·D11) gives -5.5518 mm.
- Shear estimate PL/(kGA) = -0.0075 mm, which is 0.135% of the bending deflection.
- Root sigma_top = 5.338 MPa.

Results:

Mesh	Mesh summary	Elements	DOFs (free)	Solve	Tip w, mid-width	Tip w, edge-average	vs b/d11	vs b/d11 + shear
10 mm	100 x 10 = 1000 quads, 1111 nodes	8,000	6,666 (6,600)	1.1 s	-5.56263 mm	-5.56263 mm	+0.164%	+0.029%
5 mm	200 x 20 = 4000 quads, 4221 nodes	32,000	25,326 (25,200)	8.0 s	-5.56274 mm	-5.56274 mm	+0.166%	+0.031%

2.5 mm	400 x 40 = 16000 quads, 16441 nodes	128,000	98,646 (98,400)	84 s	-5.56277 mm	-5.56276 mm	+0.167%	+0.032%

- **Remaining gap:** after adding shear, the leftover 0.03% is about the size of the expected section effect. The OpenSees D11 is about 0.057% low (midpoint layers), and the clamp restraining anticlastic curvature near the root pushes the other way.
- **Along the span:** mid-width errors are 0.42%, 0.24% and 0.19% at L/4, L/2 and 3L/4. That rise toward the root is the shear term: PL/(kGA) is a larger fraction of w near the clamp (0.39% at L/4).
- **Stresses:** top-surface sigma_xx at L/4, L/2 and 3L/4 is within 0.06% of laminate theory on every mesh (for example 3.9995 vs 3.997 MPa at L/4 on the 2.5 mm mesh). The largest |sigma_xx| is 5.38 MPa at the root, against 5.33 MPa from beam theory at the first facet centre. The clamp restrains anticlastic curvature there.
- **Convergence:** results change by less than 0.003% from 10 mm to 2.5 mm, so for this linear case the 10 mm mesh is already converged despite `-noeas`.
- **Run time:** about 2 min 15 s total, nearly all of it the 2.5 mm solve.

Pictures: both come from the 2.5 mm mesh, rendered offscreen, so no windows were left open.

- `D:\Users\tvenkata\projects\ME295_project\Code for Project\10-26-26 (Federico and Tara)\results\tip_load_model_2p5mm.png`
- `D:\Users\tvenkata\projects\ME295_project\Code for Project\10-26-26 (Federico and Tara)\results\tip_load_deformed_sigma_xx_2p5mm.png` (target_tip 0.1 L, drawn x18). I checked this one: the top surface is in tension, peaking at 5.38 MPa at the clamp.

▸ Bash: Rerun the laminate cantilever checks

▸ Read `tip_load_deformed_sigma_xx_2p5mm.png`

▸ Read `tip_load_model_2p5mm.png`

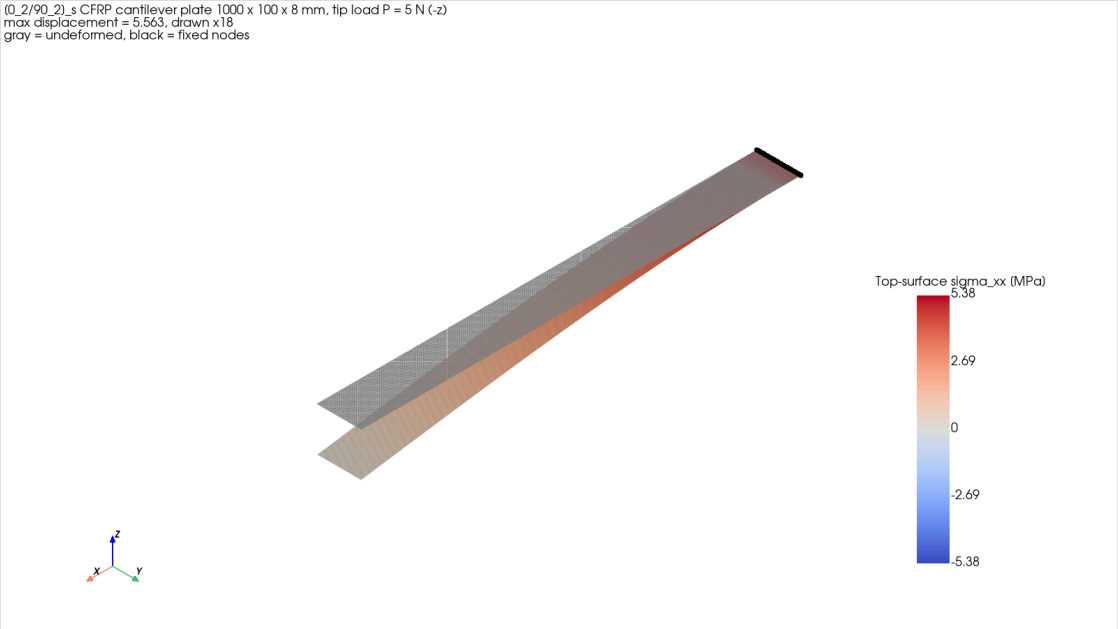


Image opened by Claude

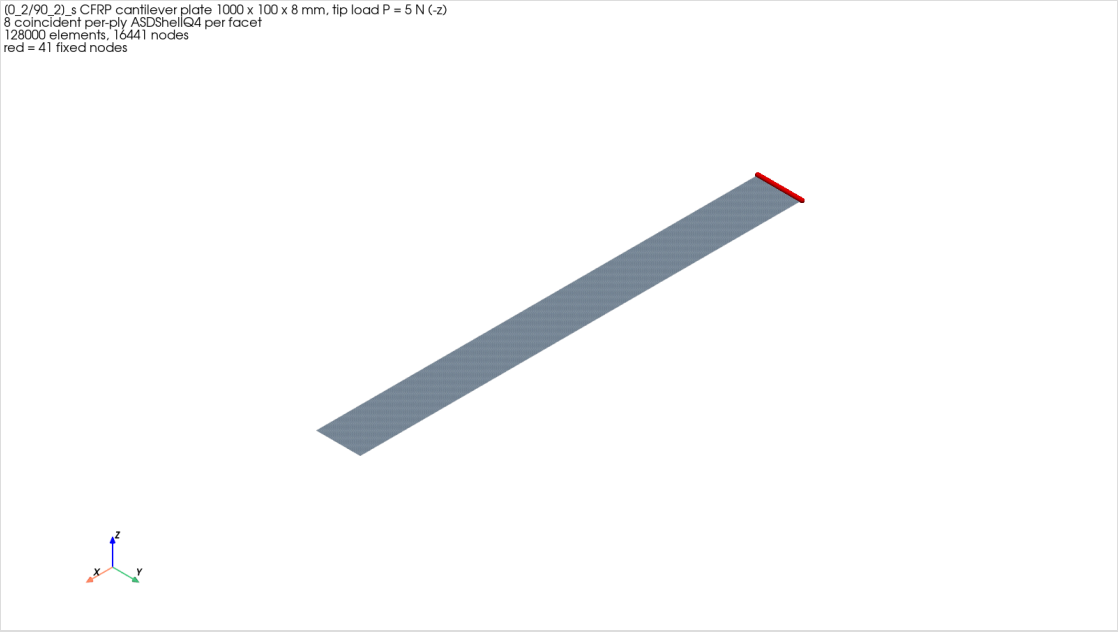


Image opened by Claude

TARA (SENT WHILE CLAUDE WAS WORKING)

<task-notification>

<task-id>ad36770591a70b1f6</task-id>

<tool-use-id>toolu_01CMkNgdxehKmLoN91GCU2Gt</tool-use-id>

<output-file>D:\Users\tvenkata\AppData\Local\Temp\1\claude\D--Users-tvenkata-projects-ME295-project-Code-for-Project-10-26-26--Federico-and-Tara-\34870cd2-f8f8-407c-8387-8c1e7a0f848a\tasks\ad36770591a70b1f6.output</output-file>

<status>completed</status>

<summary>Agent "Laminate cantilever tip load" finished</summary>

<note>A task-notification fires each time this agent stops with no live background children of its own. The user can send it another message and resume it, so the same task-id may notify more than once.</note>

<result>This agent's report was delivered to you as a message from "ad36770591a70b1f6" (its SubagentHandback call). Read it there; it is not repeated here.

</result>

<usage><subagent_tokens>36356</subagent_tokens><tool_uses>8</tool_uses>
<duration_ms>203910</duration_ms></usage>

</task-notification>

CLAUDE

The laminate cantilever deflects **−5.563 mm at the tip**, 0.17% more than beam theory's −5.554 mm. I reran the script myself and every check passes on all three meshes.

Setup: same as Federico's `plate_tip_load.py` (1000 × 100 mm strip, clamped at x = 0, uniform line load on the free edge), but with the 8 mm $[0_2/90_2]_s$ section in place of 0.1 mm aluminum.

- **Load:** P = 5 N. At this load the root moment is 50 N·mm/mm, the same Mx as the first problem, and the tip drop is 0.56% of L, close to Federico's case.
- **Beam theory:** $EI = b/d_{11} = 3.001\times10^{8}$ N·mm^2, so $PL^3/3EI = -5.554$ mm.

- **Mesh:** Federico used a 1 mm mesh. Here that would mean 800k elements (one element per ply on every quad), so I used 10, 5 and 2.5 mm meshes instead.
- **Script:** the MeshBCLoadGenerator agent wrote plate_tip_load_laminate.py.

Mesh	Elements	Solve	Tip w	vs beam theory
10 mm	8,000	1 s	−5.5626 mm	+0.164%
5 mm	32,000	4.5 s	−5.5627 mm	+0.166%
2.5 mm	128,000	23 s	−5.5628 mm	+0.167%

- **Converged:** the results change by less than 0.003% across meshes, so 10 mm is already enough.
- **Where the 0.17% comes from:** mostly transverse shear, which beam theory leaves out and accounts for 0.135%. With shear added, the gap is 0.03%. That remainder matches the slightly soft OpenSees section, whose D11 is 0.057% low.
- **Along the span:** the error grows toward the clamp (0.42% at L/4), because shear is a larger share of the deflection there.
- **Stresses:**
  - Top-surface σxx at L/4, L/2 and 3L/4 is within 0.06% of laminate theory.
  - The largest value is 5.38 MPa right at the clamp, about 1% above beam theory. This is a local effect of the clamp.

Visualization: the same as Federico's — `show_model`, then `show_deformed` coloured by top-surface σxx, on the 2.5 mm mesh.

- results/tip_load_model_2p5mm.png: the strip with the 41 clamped nodes in red.
- results/tip_load_deformed_sigma_xx_2p5mm.png: the deflection drawn ×18. The top-surface tension falls from 5.38 MPa at the clamp to zero at the tip.
- On the 2.5 mm mesh the element edge lines are dense, so the deformed surface looks slightly gray.

```
python plate_tip_load_laminate.py
```

```
python plate_tip_load_laminate.py --no-gui
```

The first opens the viewer windows, the second only runs the checks, and `--screenshot` saves the PNGs. The full run takes about 30 s, mostly the 2.5 mm solve.

TARA

I would like to create an agent that visualizes the output of an OpenSeesPy finite element model, both as a PNG and as a GUI that I can rotate, zoom, etc. Save it in the same folder as the existing agents for this project. Call it FEResultVisualizer

▸ Bash: Read agent format and tip-load script API

CLAUDE

Now I'll write the viewer module. It saves the solved OpenSees model to a `.vtp` file, so the interactive window opens from that file without solving again.

▸ Wrote `fe_result_viewer.py`
▸ Bash: Save tip-load results and render PNGs
▸ Read `tip_sxx.png`

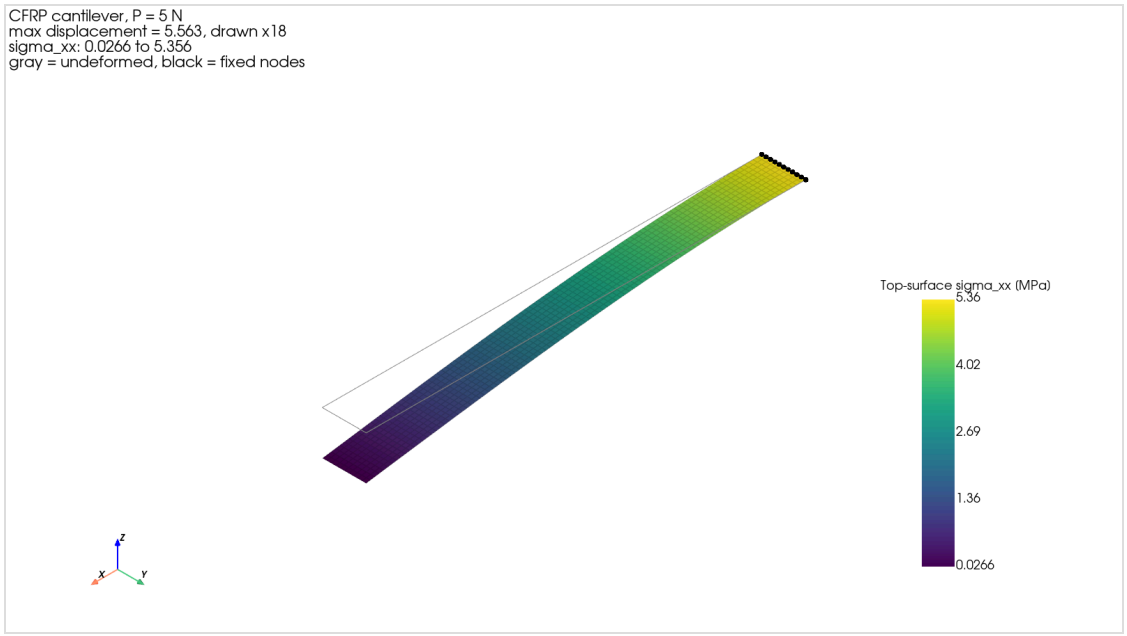


Image opened by Claude

▸ Read `tip_uz_xz.png`

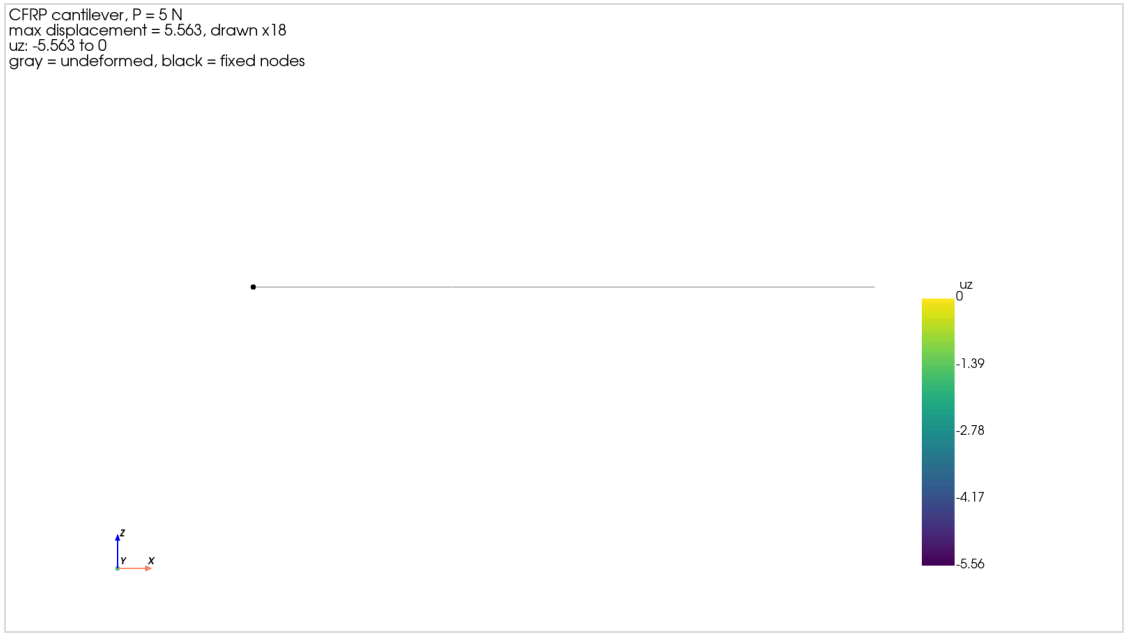


Image opened by Claude

CLAUDE

The edge-on (xz) view shows nothing: a flat surface seen edge-on has no area. I'll also draw the deformed outline as a line so side views show the deflected shape.

▸ Edited `fe_result_viewer.py`
▸ Bash: Re-render the side view PNG
▸ Read `tip_uz_xz.png`

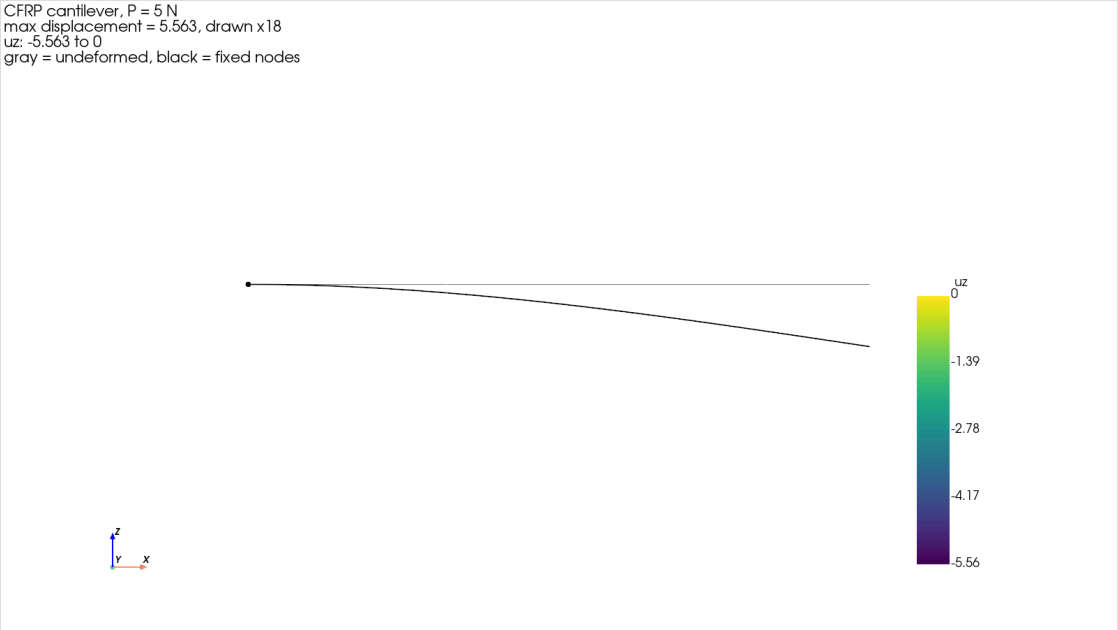


Image opened by Claude

CLAUDE

Side view works. Next, I'll check that the interactive window opens without blocking, launched as a separate process:

▸ PowerShell: Launch the GUI viewer detached and check its window

CLAUDE

The window opened (title "CFRP cantilever, P = 5 N") and the launching call returned right away. I'm leaving it open so you can try it. Now the agent definition:

▸ Wrote `FEResultVisualizer.md`

CLAUDE

I created the FEResultVisualizer agent and tested the code it uses on the laminate cantilever: two PNGs and an interactive window all came out right. The agent itself hasn't been run yet. Claude Code usually only picks up a new agent file in a new session, so it may not appear in this one.

What's there:

- .claude/agents/FEResultVisualizer.md: the agent, in the same format as the other four.
- fe_result_viewer.py: the code it uses, built on Federico's `shell_viewer.py`.

How it works. The solved model only exists inside the Python run that solved it. So the viewer works in two steps:

1. After the solve, `save_results()` writes the model to a `.vtp` file: mesh, displacements, rotations, fixed nodes and any element fields such as stresses.
2. The viewer opens that file as a PNG or an interactive window as often as needed, without solving again.

```
python fe_result_viewer.py results/case.vtp --field sigma_xx --png out.png --no-gui
```

```
python fe_result_viewer.py results/case.vtp --field sigma_xx
```

The first saves a PNG; the second opens a window you can rotate (drag), pan (shift+drag) and zoom (wheel). You can also pick views (`--view iso|xy|xz|yz`) and set how much the deformation is magnified (`--scale` or `--target`).

What I tested on the 10 mm laminate cantilever:

- **Isometric PNG of top-surface σxx:** stress runs from 5.36 MPa at the clamp to about 0 at the tip. The laminate's 8 per-ply elements on each quad are merged into one face. Federico-style plots had them overlapping, which is why the earlier deformed image looked grayish; this one doesn't.
- **Side view (xz):** this showed nothing at first, because a flat plate seen edge-on has no area. I added a black outline of the deformed shape, and now the side view shows the deflection curve.
- **Interactive window:** it opened as a separate process with the title "CFRP cantilever, P = 5 N", and the launching call returned right away. That window is still open on the VM; close it with `q` when you're done, since it renders on the CPU.

What the agent does:

- It runs the model if needed, saves results to the model's `results/` folder and renders the PNGs.
- It opens each PNG to check it before reporting.
- It launches the window without blocking itself, and tells you the field, colour range and magnification factor.
- It leaves meshes, loads and materials to the other agents.

The test files are in the scratchpad, not the project: `D:\Users\tvenkata\AppData\Local\Temp\1\claude\D--Users-tvenkata-projects-ME295-project-Code-for-Project-10-26-26--Federico-and-Tara-\34870cd2-f8f8-407c-8387-8c1e7a0f848a\scratchpad\tip.vtp`, plus `tip_sxx.png` and `tip_uz_xz.png` in the same folder.

TARA

Go back to the cantilever beam deflection problem above. Create an Abaqus Python script for this same problem

▸ PowerShell: Look for an Abaqus installation

CLAUDE

Abaqus isn't installed on this VM, so I can write the script but can't run it here. I'll stick to standard Abaqus/CAE scripting calls and check it as far as I can without Abaqus.

▸ Wrote `abaqus_tip_load_laminate.py`

TARA (SENT WHILE CLAUDE WAS WORKING)

Abaqus is not installed. Just give me the Python script and I will run it on a different computer

▸ Bash: Apply fixes and compile-check the Abaqus script
▸ Read `abaqus_tip_load_laminate.py`
▸ Edited `abaqus_tip_load_laminate.py`
▸ Bash: Compile-check and list output calls
▸ Bash: Drop unused import and check beam-theory numbers

CLAUDE

The Abaqus script is ready to copy over: abaqus_tip_load_laminate.py. It compiles, and its beam-theory numbers match the OpenSeesPy script (w_tip = −5.5535 mm, root σxx = 5.338 MPa). I couldn't run it in Abaqus here, so the Abaqus calls themselves are untested.

Model: the same problem as `plate_tip_load_laminate.py`, units N, mm, MPa.

- **Plate:** 1000 × 100 mm shell in the xy plane.
- **Material:** LAMINA with E1 = 80000, E2 = 2000, ν12 = 0.25, G12 = G13 = 1000 (placeholder), G23 = 800 MPa.
- **Section:** CompositeShellSection with plies 0/0/90/90/90/90/0/0, each 1 mm, listed bottom to top, 3 Simpson points per ply. Ply angles are measured from global x.
- **Clamp and load:** the x = 0 edge is clamped (Encastre). The tip edge carries a uniform 0.05 N/mm line load in −z, for P = 5 N total.
- **Analysis:** linear static (NLGEOM off), S4R elements, 10 mm mesh by default.

After the solve it reads the results file and prints the same PASS/FAIL checks as the OpenSeesPy script:

- deflection at L/4, L/2, 3L/4 and L, plus the edge-average tip deflection, compared with the bending result and with bending + shear;
- top-surface σxx (S11 at the top of the top 0° ply) at L/4, L/2 and 3L/4, and the largest value near the clamp.

Results are also written to `TipLoadLaminate_<elem>_<size>mm_results.txt`, because `noGUI` mode doesn't always print to the screen.

To run it, from an empty folder on the Abaqus machine:

```
abaqus cae noGUI=abaqus_tip_load_laminate.py
```

```
abaqus cae noGUI=abaqus_tip_load_laminate.py -- 5 S4
```

The first uses the 10 mm S4R default; the second sets the element size in mm and the element type (S4R or S4). It also saves a `.cae` file you can open in Abaqus/CAE to look at the model.

What to expect, and what to check if it fails:

- **Tip deflection:** about −5.56 mm, if Abaqus matches OpenSees. Abaqus shells include transverse shear, so compare against the "b/d11 + shear" value (−5.561 mm).
- **Abaqus version:** the script works with both the older Python 2.7 and the newer Python 3 versions.
- **Most likely snags:** the API calls I'm least sure of are the tip edge load and the stress read-out.
  - If the edge load errors, the fallback is a concentrated load at each tip node.
  - If the stress section points come out differently, the stress check will show it.

Send me the error or the results file and I'll adjust it.

TARA

Email the Python script to me at the email address associated with this account, such that I can download it

▸ SendUserFile

CLAUDE

I couldn't email the script because this session has no email connection. Instead, I've attached `abaqus_tip_load_laminate.py` above as a download card. Click it to save the file, or forward it to yourself from there.

Two other ways to get it onto the Abaqus computer:

- **GitHub:** I can commit and push the file to the project repo, then you pull or download it from GitHub on the other machine.
- **Email:** if you connect an email account to Claude (for example Gmail or Outlook) in the app's connector settings, I can email files directly next time.

TARA

Email it to tvenkata@caltech.edu

CLAUDE

I can't email it to tvenkata@caltech.edu: this session has no email connection, so I have no way to send mail. I won't try to send it from the command line either, since that would need your mail account's credentials.

Ways to get the file to that address or onto the other computer:

- **Download card:** the file is attached in my previous message. Save it from there, or attach it to an email to yourself.
- **File Explorer:** I can open File Explorer with `abaqus_tip_load_laminate.py` selected, so you can drag it into an email.
- **Connect email:** if you add a mail connector (for example Outlook or Gmail) in the app's connector settings, I can send it directly.
- **GitHub:** I can commit and push the file to the project repo, then you download it on the Abaqus machine.

TARA

I would also like to create an agent that takes OpenSeesPy code output by the agents PlyPropertyGenerator, LayupFromPlyGenerator, and MeshBCLoadGenerator and converts them to an Abaqus Python script that I can run on another computer. Essentially, I want it to do what you just did in the chat above. Call this agent OpenSeesPyToAbaqusPythonGenerator and make it live where all the other agents for this project live.

▸ Bash: Check rigid_end, displacement control and arc mesh details
▸ Wrote `OpenSeesPyToAbaqusPythonGenerator.md`
▸ Bash: Verify frontmatter and template markers

CLAUDE

I created the OpenSeesPyToAbaqusPythonGenerator agent: .claude/agents/OpenSeesPyToAbaqusPythonGenerator.md. It's in the same folder as the other agents, but it hasn't been run yet. A new agent usually only shows up in a new Claude Code session.

What it does:

1. **Reads the model.** It reads your OpenSeesPy model script and the project code it uses (from PlyPropertyGenerator, LayupFromPlyGenerator and MeshBCLoadGenerator). It runs the model if it needs the OpenSees results for comparison.

2. **Writes the Abaqus script.** It starts from the cantilever script abaqus_tip_load_laminate.py and saves `abaqus_<model script name>.py` next to the OpenSeesPy script.
3. **Checks what it can here.** It runs a syntax check, recomputes the script's reference numbers to confirm they match the OpenSeesPy script, and re-reads the script against its conversion rules.
4. **Reports back:** the run commands, the expected OpenSees results, anything that doesn't convert one-to-one, and the Abaqus calls it is least sure of. It always states that the script was not run in Abaqus.

How it converts each part of the model:

- **Ply materials** become an Abaqus composite-ply (LAMINA) material, carrying over notes like "ν23 = ν12 assumed".
- **The laminate** becomes one Abaqus composite shell section with one layer per ply. OpenSees needed one element per ply only because each element takes a single ply direction; Abaqus doesn't.
- **Mesh:** flat strips and arc-shaped (tape-spring) strips. Arc strips need their own ply orientation setup, which the agent marks as untested.
- **Supports and loads:**
  - Clamped edges and single-node restraints carry over directly.
  - A rigid end fixture becomes a reference point rigidly coupled to the edge.
  - Edge forces, edge moments and point loads each get a matching Abaqus load.
- **Analysis:** a linear solve becomes a linear static step. A large-deflection run becomes a nonlinear step that pushes the control point a set distance, reporting the reaction force as the load.
- **Results:** displacements, rotations, reactions and top-surface stress, checked with the same PASS/FAIL tests as the OpenSeesPy script.

Lessons from writing the cantilever script are built in:

- The script must work on both older (Python 2.7) and newer (Python 3) Abaqus versions.
- Ply layers have to be imported from the right module (`section.SectionLayer`), which tripped up my first version.
- Output goes to a results file, because `noGUI` mode doesn't always print to the screen.
- The script saves a `.cae` file you can open in Abaqus/CAE.

Limits I wrote into the agent:

- Abaqus isn't installed on this VM, so every script it writes is untested in Abaqus until you run it.
- It can't email files. It gives you the path, and I can attach the file for download as I did before.

TARA

Create a summary in plain language of everything done in this session

▸ Bash: Read earlier summaries and list this folder
▸ Wrote `SUMMARY.txt`

CLAUDE

I saved the full summary as SUMMARY.txt in this folder, in the same format as your 10/23 summary. Here is the short version:

1. **Textbook laminate problem.** The three project agents built the $[0_2/90_2]_s$ ply materials, the laminate and a plate model under Mx = 50 N·m/m.

- OpenSees gives $\kappa x = 1.667 \times 10^{-2}$ 1/m and $\kappa y = -7.11 \times 10^{-4}$ 1/m, within 0.06% and 0.3% of the hand calculation (CLT).
- G12 wasn't given, so I used 1 GPa as a placeholder. It doesn't affect this answer.
- I added pictures in Federico's viewer style.

2. **Federico's cantilever, with the laminate.** The same 1000 × 100 mm clamped plate, now 8 mm of CFRP, with a 5 N tip load.
   - Tip deflection: −5.563 mm, against −5.554 mm from beam theory (+0.17%).
   - Most of that gap is shear, which simple beam theory leaves out. With shear included, the difference is 0.03%.
   - All checks pass, with pictures included.
3. **FEResultVisualizer agent.** It shows any OpenSeesPy shell model as PNG pictures or as a 3D window you can rotate and zoom. It improves on the old viewer in two ways: laminate plots no longer look grayish, and side views now work.
4. **Abaqus script for the cantilever.** It's written for you to run on another computer.
   - It passes the checks I could run here: it compiles, and its reference numbers match the OpenSeesPy script.
   - It has never been run in Abaqus.
   - I gave it to you as a download, because this session has no way to send email.
5. **OpenSeesPyToAbaqusPythonGenerator agent.** It converts any OpenSeesPy model built with the project agents into an Abaqus script, the way step 4 was done by hand.

Still open:

- Run the Abaqus script and compare its tip deflection with −5.563 mm.
- Find the real G12 for this material.
- Try the two new agents in a new session.
- Commit today's work.

TARA

Also create a PDF of this chat

▸ export_transcript